

383. Ransom Note Easy

Acceptance: 65.5% | [LeetCode](#)

Given two strings `ransomNote` and `magazine`, return `true` if `ransomNote` can be constructed by using the letters from `magazine` and `false` otherwise.

Each letter in `magazine` can only be used once in `ransomNote`.

Example 1:

```
**Input:** ransomNote = "a", magazine = "b"  
**Output:** false
```

Example 2:

```
**Input:** ransomNote = "aa", magazine = "ab"  
**Output:** false
```

Example 3:

```
**Input:** ransomNote = "aa", magazine = "aab"  
**Output:** true
```

Constraints:

- `1 <= ransomNote.length, magazine.length <= 105`
- `ransomNote` and `magazine` consist of lowercase English letters.

Claude Opus Solution

Ransom Note - Solution Explanation

1. Intuition

The **key insight** is that this is fundamentally a **counting/frequency problem**.

To build `ransomNote` from `magazine`, we need enough of *each* letter. Since each letter in the magazine can only be used once, we essentially need:

For every character, the count in `magazine` must be **greater than or equal to** its count in `ransomNote`.

The order of letters doesn't matter—only the **quantity** of each letter. This immediately points us toward a **hash table (frequency map)** pattern.

2. Approach (Pseudocode)

1. Build a frequency map of all characters in `magazine`.
2. For each character in `ransomNote`:
 - a. If the available count for that character is 0, we can't build the note -> return `False`.
 - b. Otherwise, "consume" one of that character (decrement count).
3. If we get through the entire note, return `True`.

3. Python Solution

```
from collections import Counter  
  
class Solution:  
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:  
        # Step 1: Count how many of each letter the magazine provides.  
        # Counter creates a dict like {'a': 2, 'b': 1, ...}  
        available = Counter(magazine)  
  
        # Step 2: Walk through every letter we NEED for the ransom note.  
        for char in ransomNote:  
            # If we have run out of this letter (count is 0 or it never existed),  
            # we cannot complete the note.  
            # Note: Counter returns 0 for missing keys, so this check is safe.  
            if available[char] <= 0:  
                return False  
  
            # "Use up" one copy of this letter so it can't be reused.  
            available[char] -= 1
```

```
# Step 3: We successfully matched every required letter.
return True
```

Early-Exit Optimization

A quick check we can add: if `ransomNote` is longer than `magazine`, it's impossible.

```
if len(ransomNote) > len(magazine):
    return False
```

Cleanest One-Liner Variant (using Counter subtraction)

```
from collections import Counter

class Solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        # Counter(ransomNote) - Counter(magazine) keeps only POSITIVE counts,
        # i.e. letters where the note needs MORE than the magazine has.
        # If the result is empty, every requirement was satisfied.
        return not (Counter(ransomNote) - Counter(magazine))
```

4. Complexity Analysis

Let $n = \text{len}(\text{ransomNote})$ and $m = \text{len}(\text{magazine})$.

Metric	Complexity	Explanation
Time	$O(n + m)$	We do one pass over <code>magazine</code> (m) to build the map, and one pass over <code>ransomNote</code> (n) to verify. Dictionary lookups/updates are $O(1)$ on average.
Space	$O(1)$	The frequency map holds at most 26 keys (lowercase English letters). This is constant regardless of input size. More generally it's $O(k)$ where k is the alphabet size.

5. Alternative Approaches

a) Fixed-size array instead of a hash map Since input is limited to 26 lowercase letters, we can use an array of length 26 indexed by `ord(c) - ord('a')`. This avoids hash overhead and is the fastest in practice.

```
class Solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        counts = [0] * 26
        # Add up magazine letters
        for c in magazine:
            counts[ord(c) - ord('a')] += 1
        # Subtract ransom note letters; any negative means a shortage
        for c in ransomNote:
            idx = ord(c) - ord('a')
            counts[idx] -= 1
            if counts[idx] < 0:
                return False
        return True
```

b) Sorting Sort both strings and use two pointers to match. This works but is slower: $O(n \log n + m \log m)$ time. Not recommended here since counting is strictly better.

c) Built-in Counter subtraction (shown above) Most Pythonic and concise, but creates intermediate Counter objects.

Pattern Takeaway

When a problem asks whether one collection can be "formed/built" from another and **order doesn't matter**, reach for a **frequency counter**. This same pattern applies to anagram checks, "minimum window," and many string-matching problems.

Codex Solution

1. Intuition

We need to know whether every character needed by `ransomNote` exists enough times in `magazine`.

This is a classic **frequency counting / hash table** problem.

Key idea:

- Count how many times each letter appears in `magazine`.
- Then try to "spend" those letters to build `ransomNote`.
- If at any point a needed letter is unavailable, return `False`.
- Otherwise, return `True`.

Since all characters are lowercase English letters, there are only 26 possible characters, so the extra space is constant.

2. Approach

Pseudocode

```
function canConstruct(ransomNote, magazine):  
    create frequency map counts  
  
    for each character ch in magazine:  
        counts[ch] += 1  
  
    for each character ch in ransomNote:  
        if counts[ch] == 0:  
            return false  
        counts[ch] -= 1  
  
    return true
```

3. Python Solution

```
from collections import defaultdict  
  
class Solution:  
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:  
        # Dictionary to store how many times each character appears in magazine.  
        # defaultdict(int) gives a default value of 0 for missing characters.  
        char_count = defaultdict(int)  
  
        # Count every character available in magazine.  
        for char in magazine:  
            char_count[char] += 1  
  
        # Try to build ransomNote using the counted characters.  
        for char in ransomNote:  
            # If this character is not available anymore,  
            # then ransomNote cannot be constructed.  
            if char_count[char] == 0:  
                return False  
  
            # Use one occurrence of this character.  
            char_count[char] -= 1  
  
        # If we successfully used characters for the entire ransomNote,  
        # then construction is possible.  
        return True
```

4. Complexity Analysis

Let:

- `n = len(ransomNote)`
- `m = len(magazine)`

Time Complexity

$O(n + m)$

Explanation:

- We scan `magazine` once to count characters: $O(m)$
- We scan `ransomNote` once to check and consume characters: $O(n)$

So total time is:

$O(n + m)$

Space Complexity

$O(1)$

Explanation:

- Since the strings only contain lowercase English letters, there are at most 26 possible keys in the dictionary.
- Therefore, the space usage is constant.

If the character set were not fixed, the space complexity would be:

$O(k)$

where `k` is the number of distinct characters in `magazine`.

5. Alternative Approaches

Alternative 1: Using `collections.Counter`

Python's `Counter` can make the solution shorter.

```
from collections import Counter

class Solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        return not (Counter(ransomNote) - Counter(magazine))
```

Explanation:

- `Counter(ransomNote)` counts needed letters.
- `Counter(magazine)` counts available letters.
- Subtracting removes available letters from needed letters.
- If anything remains, then `ransomNote` cannot be constructed.

Time complexity is still:

$O(n + m)$

Space complexity is:

$O(1)$

because there are only 26 lowercase letters.

Alternative 2: Fixed-size Array of Length 26

Since all letters are lowercase English letters, we can use an array instead of a dictionary.

```
class Solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        counts = [0] * 26

        for char in magazine:
            index = ord(char) - ord('a')
            counts[index] += 1

        for char in ransomNote:
            index = ord(char) - ord('a')

            if counts[index] == 0:
                return False

            counts[index] -= 1
```

```
return True
```

This is slightly faster in practice because array indexing is cheaper than dictionary lookup.

Time complexity:

```
O(n + m)
```

Space complexity:

```
O(1)
```

Alternative 3: Sorting

We could sort both strings and compare character usage, but this is less efficient.

Time complexity:

```
O(n log n + m log m)
```

This is worse than the frequency counting solution, so it is not preferred.